1) [14 pts] Consider the following table with the given assumptions.

Student (StudentID, FirstName, LastName, Gender, Age, DepartmentID)

Assumptions:

- There is no index.
- There are total 150,000 rows.
- 2,000 rows fall within the 35 to 40 age range.
- There are 40 rows for each page.
- 100 index rows fit on a non-clustered index leaf page.

What is the number of page accesses of the following queries for the following cases with the given assumptions? Consider each case separately.

a) [7 pts] Create Nonclustered Index Ix1 on Student(Age);

Select StudentID From Student Where Age>=35 and Age<40 and Gender='F';

b) [7 pts] Create Nonclustered Index Ix2 on Student(StudentID,Age);

Select StudentID From Student Where Age>=35 and Age<=40;

"

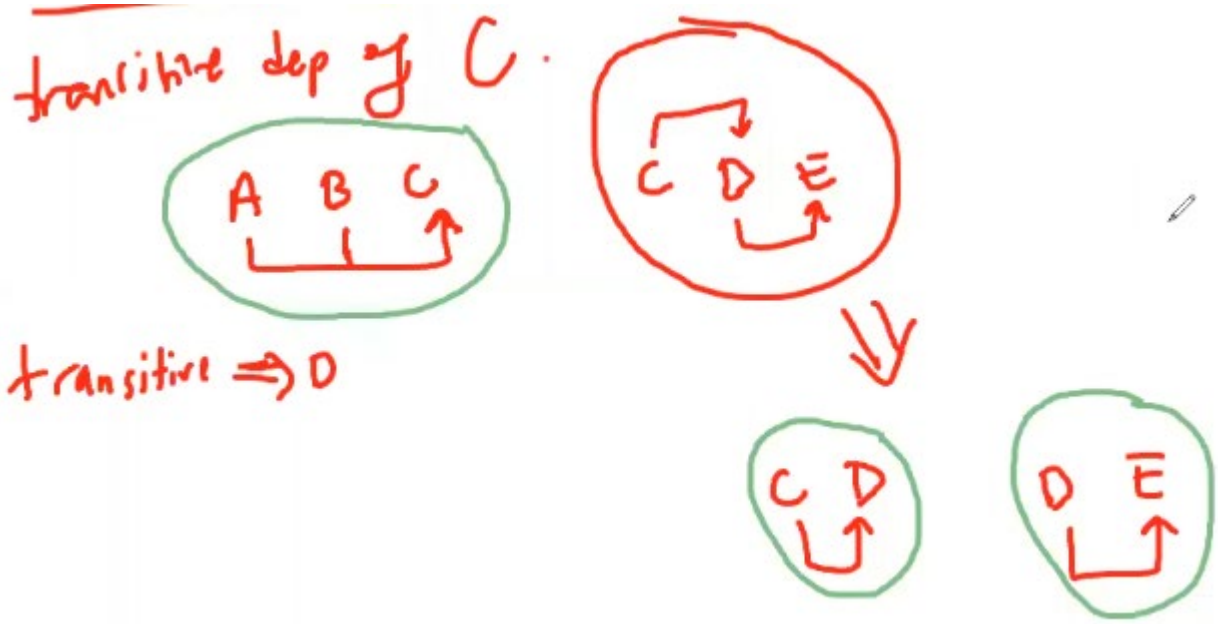
2) [14 pts] Suppose you are given a relation *R* with five attributes: *R = {A, B, C, D, E}*. We have the following functional dependencies:

- {A, B} ⇨ {C}
- {C} ⇨ {D}
- {D} ⇨ {B}
- {D} ⇨ {E}

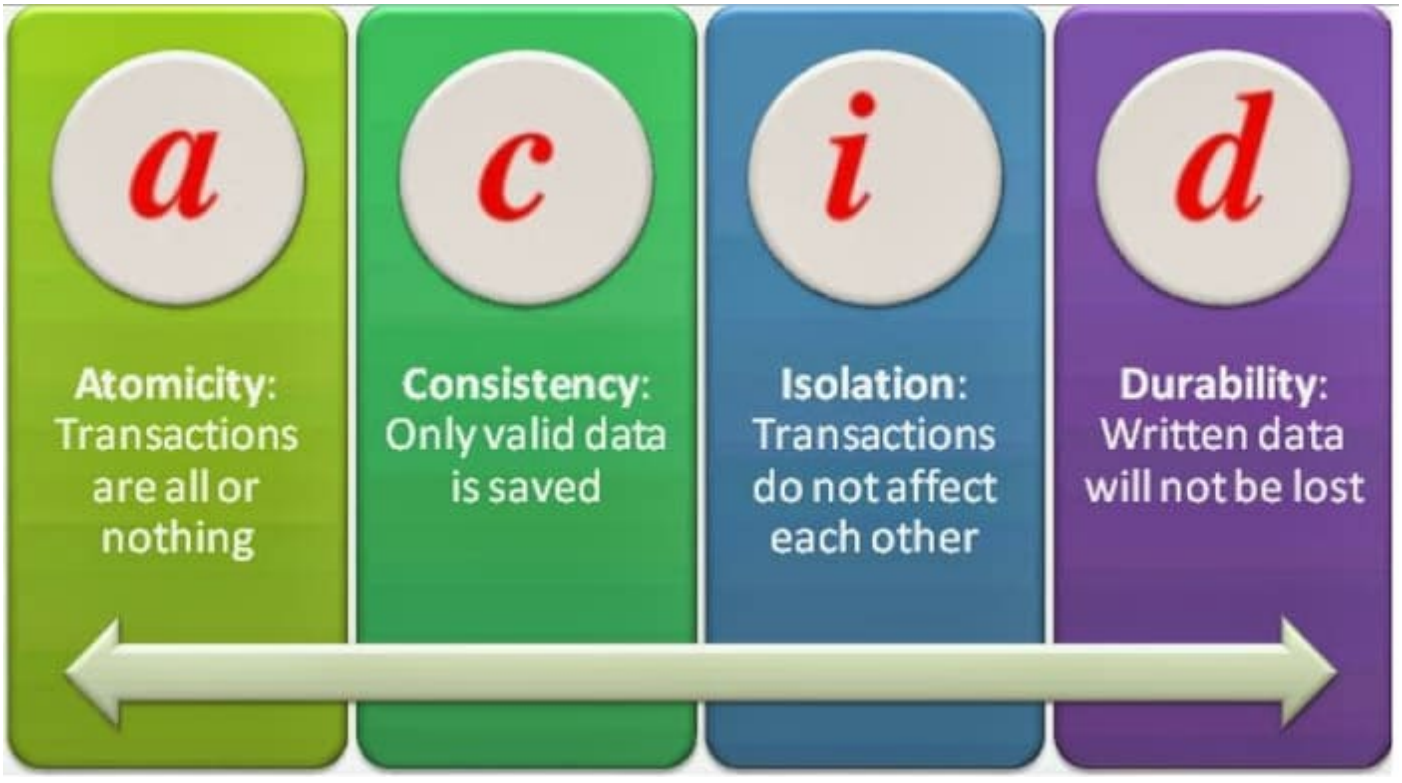
a) [3 pts] Identify the candidate key(s) for *R*.

b) [4 pts] Identify the best normal form that *R* satisfies (1NF, 2NF, 3NF). Explain.

c) [7 pts] If *R* is not in 3NF, decompose it into a set of 3NF relations step by step (1NF → 2NF → 3NF) that preserve the dependencies.



3) [8 pts] List and briefly explain the ACID properties of a database transaction.



*** Use ***Site Management*** database below to write the following SQL queries.

Person

PersonID	FirstName	LastName	Birthdate	Email	Tel
Int	*nvarchar(50)*	*nvarchar(50)*	*smalldatetime*	*varchar(50)*	*varchar(50)*
1	Ali	AYDIN	1970-01-01	ali@aydin.com	0532 787 99 88
2	Hakan	YILMAZ	1965-01-01	hakan@gmail.com	0542 444 77 88
7	Mustafa	AĞAOĞLU	1974-01-01	agaoglum@gmail.com	0532 222 22 22
8	Mahmut	NEYZEN	1944-01-01	mahmut@outlook.com	0555 666 44 11
..	..	..	..	..	..

Flat

FlatID	OwnerID	RenterID	AreaM2	TotalAmountToBePaid	TotalAmountPaid
Int	*int*	*int*	*decimal(7,2)*	*decimal(10,2)*	*decimal(10,2)*
1	1	1	90.00	0.00	0.00
2	2	NULL	85.00	0.00	0.00
32	8	8	100.00	0.00	0.00
41	7	7	225.00	0.00	0.00
..	..	..	..	..	..

SiteExpense

ExpenseType	PeriodID	AmountToBePaid	PaymentDueDate	AmountPaid	DatePaid
varchar(15)	*char(7)*	*decimal(10,2)*	*smalldatetime*	decimal(10,2)	*smalldatetime*
Electricity	2016-11	3833.70	2016-12-15	3833.70	2016-12-15
Heating	2016-11	4500.85	2016-12-25	450.85	2016-12-20
Heating	2016-12	5479.00	2017-01-18	NULL	NULL
Water	2016-11	2528.00	2016-12-20	2528.00	2016-12-20
..	..	..	..	..	..

SiteIncome

IncomeType	PeriodID	FlatID	FirstIndex	LastIndex	AmountToBePaid	PaymentDueDate	AmountPaid	DatePaid
varchar(15)	*char(7)*	*int*	*decimal(10,2)*	*decimal(10,2)*	*decimal(10,2)*	*smalldatetime*	*decimal(10,2)*	*smalldatetime*
Electricity	2016-11	2	81.95	81.95	0.00	2016-12-10	0.00	2016-12-10
Heating	2016-12	41	6254.00	7215.00	154.41	2017-01-10	NULL	NULL
Water	2016-11	32	83.40	90.50	33.59	2016-12-20	33.59	2016-12-19
..	..	..	..	..	..	..	..	..

4) [12 pts] Retrieve full name (first name and last name concatenated with a blank) of the owner, full name (first name and last name concatenated with a blank) of the renter, flat ID, and area of the flat for the flats who have amounts to be paid (use field *AmountToBePaid* in table *SiteIncome*) more than 49 Turkish Liras and 99 kuruş (cent) with the index difference less than 10 in the period 2016-12. Rename the column name for owner and renter as "Owner's Name" and "Renter's Name", respectively. Have your list ordered by flat ID. Keep in mind that some flats may not have renters.

```
Select po.FirstName + ' ' + po.LastName [Owner's Name],
  pr.FirstName + ' ' + pr.LastName "Renter's Name",
  f.FlatID, f.AreaM2
From Flat f inner join Person po on f.OwnerID=po.PersonID
  left outer join Person pr on f.RenterID=pr.PersonID
  inner join SiteIncome si on f.FlatID=si.FlatID
Where si.AmountToBePaid>49.99 and (si.LastIndex-si.FirstIndex)<10 and
  si.PeriodID='2016-12'
Order By f.FlatID
```

5) [16 pts] Retrive flat ID, the last name of the owner, the percentage of the amount paid to the amount to be paid, and the cluster (Platinum, Gold, Bronze) for the flats which have at least 5 income records (consider the fields *AmountToBePaid* and *AmountPaid* in table *SiteIncome*). If the percentage is greater than 75, the cluster value will be Platinum; if it is between 75 and 50, the cluster will be Gold; and if it is less than 50, then the cluster will be Bronze. If any, discard redundant data.

```
Select f.FlatID, p.LastName, fs.PercentPaid,
  IIF(fs.PercentPaid>75, 'Platinum', iif(fs.PercentPaid>=50, 'Gold', 'Bronze')) Cluster
From flat f inner join Person p on f.OwnerID=p.PersonID
  inner join
    (Select si.FlatID, 100*sum(si.AmountPaid)/sum(si.AmountToBePaid) PercentPaid
     From SiteIncome si
     Group By si.FlatID
     Having count(*)>=5) fs
  on f.FlatID=fs.FlatID
```

6) [20 pts] Implement a stored procedure such that:

- It takes one input parameter: a period (*i.e.* month).
- Given a period, calculate the heating payment for the flats, *i.e.* update the records in table *SiteIncome* for the given period with the following assumptions:
  - There are totally 44 flats starting from 1 to 44. You may use table *Flat*.
  - Calculations will be done only for heating expense/income type for the given period. *Income/Expense Type* will be "Heating".
  - Fixed Cost (FC): For the given period, 30% of the amount (*AmountToBePaid)* of the expense will be distributed to the flats according to their areas (*AreaM2*).
  - Variable Cost (VC): For the given period, 70% of the amount (*AmountToBePaid*) of the expense will be distributed to the flats according to their Index differences (*LastIndex-FirstIndex*).
  - Total Cost, which is the amount to be paid for the flat, is equal to the sum of FC and VC.
  - You have the related records in table *Income* before your calculations. For the given period, the fields *IncomeType*, *PeriodID*, *FlatID*, *FirstIndex*, and *LastIndex* are not null (there are values stored in those fields), and you are going to calculate the field *AmountToBePaid*.

```
Create Procedure sp_UpdateHeatingSiteIncome
  @period char(7)
As
Begin

  --FC %30 -> area
  Declare @totalArea decimal(7,2)
  set @totalArea = (Select sum(f.AreaM2) From Flat f)

  Declare @totalIndex decimal(10, 2)
  Set @totalIndex = (Select sum(si.LastIndex-si.FirstIndex)
                     From SiteIncome si
                     Where si.PeriodID=@period
                       and si.IncomeType='Heating')

  Update si
  Set si.AmountToBePaid = 0.30*se.AmountToBePaid
                              *f.AreaM2/@totalArea
                        + 0.70*se.AmountToBePaid
                              *(si.LastIndex-si.FirstIndex)
                              /@totalIndex
  From SiteIncome si inner join SiteExpense se
    on (se.ExpenseType=si.IncomeType and se.PeriodID=si.PeriodID)
    inner join Flat f on si.FlatID=f.FlatID
  Where se.ExpenseType='Heating' and se.PeriodID=@period
End
```

7) [16 pts] Implement a trigger such that:

- It will be fired whenever there is an update on table *SiteIncome*.
- When the update operation takes place, you will update the fields *TotalAmountToBePaid* and *TotalAmountPaid* in table *Flat* for those flats only.

```
Create Trigger trg_UpdateFlatPayments
  on SiteIncome
  after update
As
Begin
  Update f
  Set f.TotalAmountToBePaid=sif.atp, f.TotalAmountPaid=sif.ap
  From Flat f inner join
    (Select si.FlatID, sum(si.AmountToBePaid) atp, sum(si.AmountPaid) ap
     From SiteIncome si inner join inserted i on si.FlatID=i.FlatID
     Group By si.FlatID) sif
     on f.FlatID=sif.FlatID

  Update f
  Set f.TotalAmountToBePaid=sif.atp, f.TotalAmountPaid=sif.ap
  From Flat f inner join
    (Select si.FlatID, sum(si.AmountToBePaid) atp, sum(si.AmountPaid) ap
     From SiteIncome si inner join deleted d on si.FlatID=d.FlatID
     Group By si.FlatID) sif
     on f.FlatID=sif.FlatID


End
```